

PROJECT REPORT: PERCEPTRON USING OOP

Team- Team++

Members-

1. Jyotiraditya Parida	BA2025017
2. Naman Bhandari	BA2025024
3. Skand Paliwal	BA2025048
4. Kartikeya Balonda	IE2025016

GitHub Link- <https://github.com/JyotiradityaParida/MiniPerceptron>

1. Project Description

This project uses simple C++ OOP to demonstrate neural networks learning Boolean logic gates from scratch.

It uses a Perceptron for learning the AND and OR gates and a MultiLayerPerceptron for handling XOR and other truth-table patterns.

For each input, the network combines the input values with its weights and bias to produce an output. The result is processed using either Step or Sigmoid activation. The project also includes Perceptron Error and Cross Entropy loss classes for calculating the difference between the predicted and expected outputs.

The weights and biases are initialized with random values and are gradually adjusted during training. The training is performed over multiple epochs, with the model checking its predictions after each epoch. The multilayer model uses a hidden layer so that it can learn patterns such as XOR that cannot be represented by a single perceptron.

The project is organized into separate classes for activation functions, loss functions, datasets, perceptron models and training. This structure helps demonstrate important OOP concepts including encapsulation, abstraction, inheritance, polymorphism, composition, constructors, destructors and function overriding.

2. Class Structure

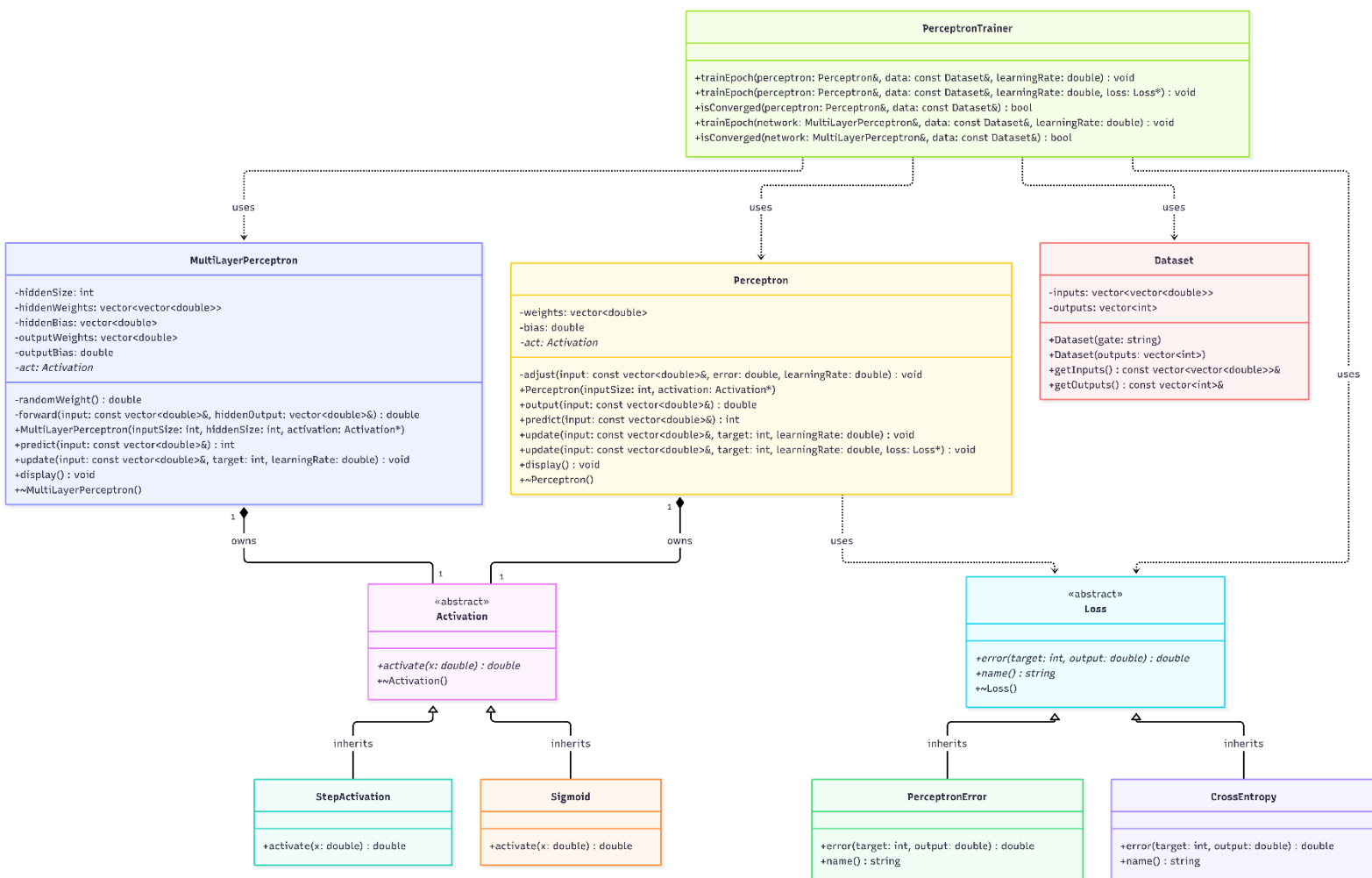
Class Name	Attributes and Methods
Activation (Abstract Base Class)	Attributes: None Methods: - activate(double x) - ~Activation()
StepActivation (Inherits from Activation class)	Attributes: None Methods: - activate(double x)
Sigmoid (Inherits from Activation class)	Attributes: None Methods: - activate(double x)
Loss (Abstract Base Class)	Attributes: None Methods: - error(int target, double output) - name() - ~Loss()
PerceptronError	Attributes: None

(Inherits from Loss class)	Methods: - error(int target, double output) - name()
CrossEntropy (Inherits from Loss class)	Attributes: None Methods: - error(int target, double output) - name()
Dataset	Attributes: - inputs - outputs Methods: - Dataset(string gate) - Dataset(vector<int> outputs) - getInputs() - getOutputs()
Perceptron	Attributes: - weights - bias - act Methods: - Perceptron(int inputSize, Activation* activation) - output(const vector<double>& input) - predict(const vector<double>& input) - adjust(const vector<double>& input, double error, double learningRate)

	- update(const vector<double>& input, int target, double learningRate) - update(const vector<double>& input, int target, double learningRate, Loss*) - display() - ~Perceptron()
MultiLayerPerceptron	Attributes: - hiddenSize - hiddenWeights - hiddenBias - outputWeights - outputBias - act Methods: - randomWeight() - forward(const vector<double>& input, vector<double>& hiddenOutput) - MultiLayerPerceptron(int inputSize, int hiddenSize, Activation* activation) - predict(const vector<double>& input) - update(const vector<double>& input, int target, double learningRate) - display() - ~MultiLayerPerceptron()
PerceptronTrainer	Attributes: None Methods: - trainEpoch(Perceptron&, const Dataset&, double)

	- trainEpoch(Perceptron&, const Dataset&, double, Loss*) - isConverged(Perceptron&, const Dataset&) - trainEpoch(MultiLayerPerceptron&, const Dataset&, double) - isConverged(MultiLayerPerceptron&, const Dataset&)
--	---

3. UML Class Diagram



5. Project Files

i. **Activation.cpp** – Implements the Step and Sigmoid activation functions and provides the actual behaviour for the activate() methods.

ii. **Activation.h** – Declares the Activation, StepActivation and Sigmoid classes.

Defines the common activation function interface.

iii. **Loss.cpp** – Implements the PerceptronError and CrossEntropy loss functions and provides their error calculations.

iv. **Loss.h** – Declares the abstract Loss class along with the PerceptronError and CrossEntropy classes.

v. **Dataset.cpp** – Creates the truth-table data for the selected AND, OR, or XOR gate and supports custom output patterns.

Implements methods for accessing the input and expected output data.

vi. **Dataset.h** – Declares the Dataset class and its attributes.

Defines constructors and methods for accessing the inputs and outputs.

vii. **Perceptron.cpp** – Implements single-layer perceptron initialization, prediction, output calculation and updating.

Also displays the current weights and bias.

viii. **Perceptron.h** – Declares the Perceptron class and its attributes.

Defines methods for prediction, updating, adjustment and displaying the perceptron.

ix. **PerceptronTrainer.cpp** – Implements the training process for both the Perceptron and MultiLayerPerceptron.

Also checks whether the models have converged.

- x. **PerceptronTrainer.h** – Declares the PerceptronTrainer class. Defines the methods used for training and convergence checking of both models.
- xi. **main.cpp** – Contains the main program and user menus. Handles gate selection, activation and loss selection, XOR and custom output selection, predictions and training interaction.
- xii. **Makefile** – Contains commands for compiling all project files and also provides the make clean command to remove generated files.
- xiii. **README.md** – Provides an overview of the project. Contains features, requirements and setup instructions.

5. How the Project Works

- i. **Program starts** – main.cpp displays the menu and asks the user to select the **AND, OR, XOR, or Custom Output** option.
- ii. **Select activation and loss** – For the Perceptron, the user selects either Step Activation or Sigmoid Activation and the available loss. The selected activation and loss objects are used during training.
- iii. **Dataset creation** – The Dataset class creates the four possible input combinations: (0,0), (0,1), (1,0) and (1,1), along with the expected outputs for the selected gate or custom output pattern.
- iv. **Model initialization** – The Perceptron initializes its weights and bias with random values and stores the selected activation function. The MultiLayerPerceptron initializes the weights and biases for its hidden and output layers.
- v. **Prediction** – The Perceptron calculates the weighted sum of the inputs, weights and bias. This value is passed through the activation function to produce the prediction.

vi. **Multi-layer prediction** – The MultiLayerPerceptron first processes the inputs through the hidden layer. The hidden-layer output is then passed to the output layer to produce the final prediction.

vii. **Compare with expected output** – The predicted output is compared with the expected output stored in the Dataset. The selected loss function calculates the error for training.

viii. **Update weights and bias** – The Perceptron updates its weights and bias using the error and learning rate. The MultiLayerPerceptron updates the weights and biases of its hidden and output layers.

ix. **Training by epochs** – PerceptronTrainer processes all four input combinations once in an epoch. The user can either view epochs one by one or display all epochs until convergence.

x. **Check convergence** – After each epoch, isConverged() checks whether all predictions match their expected outputs. The multi-layer network also stops when its maximum epoch limit (10000) is reached.

xi. **Final result** – When all predictions are correct, the model is considered converged. The final weights, biases and predictions are displayed. If the maximum epoch limit (10000) is reached, the program reports that the maximum number of epochs has been reached.

6. OOP Concepts Used

OOP Concept	How it was used in the project
Encapsulation	Data members such as weights, bias, inputs, outputs, hidden weights and hidden biases are kept private and accessed through public methods.
Abstraction	Activation and Loss are abstract classes that define common interfaces through the pure virtual functions <code>activate()</code> and <code>error()</code> .
Inheritance	StepActivation and Sigmoid inherit from the Activation base class. PerceptronError and CrossEntropy inherit from the Loss base class.
Polymorphism	Activation* and Loss* pointers allow different activation and loss objects to be used through their common interfaces.
Function Overriding	StepActivation and Sigmoid provide their own implementations of <code>activate()</code> . PerceptronError and CrossEntropy provide their own implementations of the loss functions.
Composition / Has-a Relationship	Perceptron and MultiLayerPerceptron have an Activation* member, meaning they use an activation object as part of their operation. Perceptron can also use a Loss* object during training.
Constructors	Constructors initialize objects, such as creating the required dataset and initializing the weights

	and biases of the Perceptron and MultiLayerPerceptron.
Destructors	The Perceptron and MultiLayerPerceptron destructors release their dynamically allocated activation objects.
Getters	getInputs() and getOutputs() provide controlled access to the private dataset attributes.